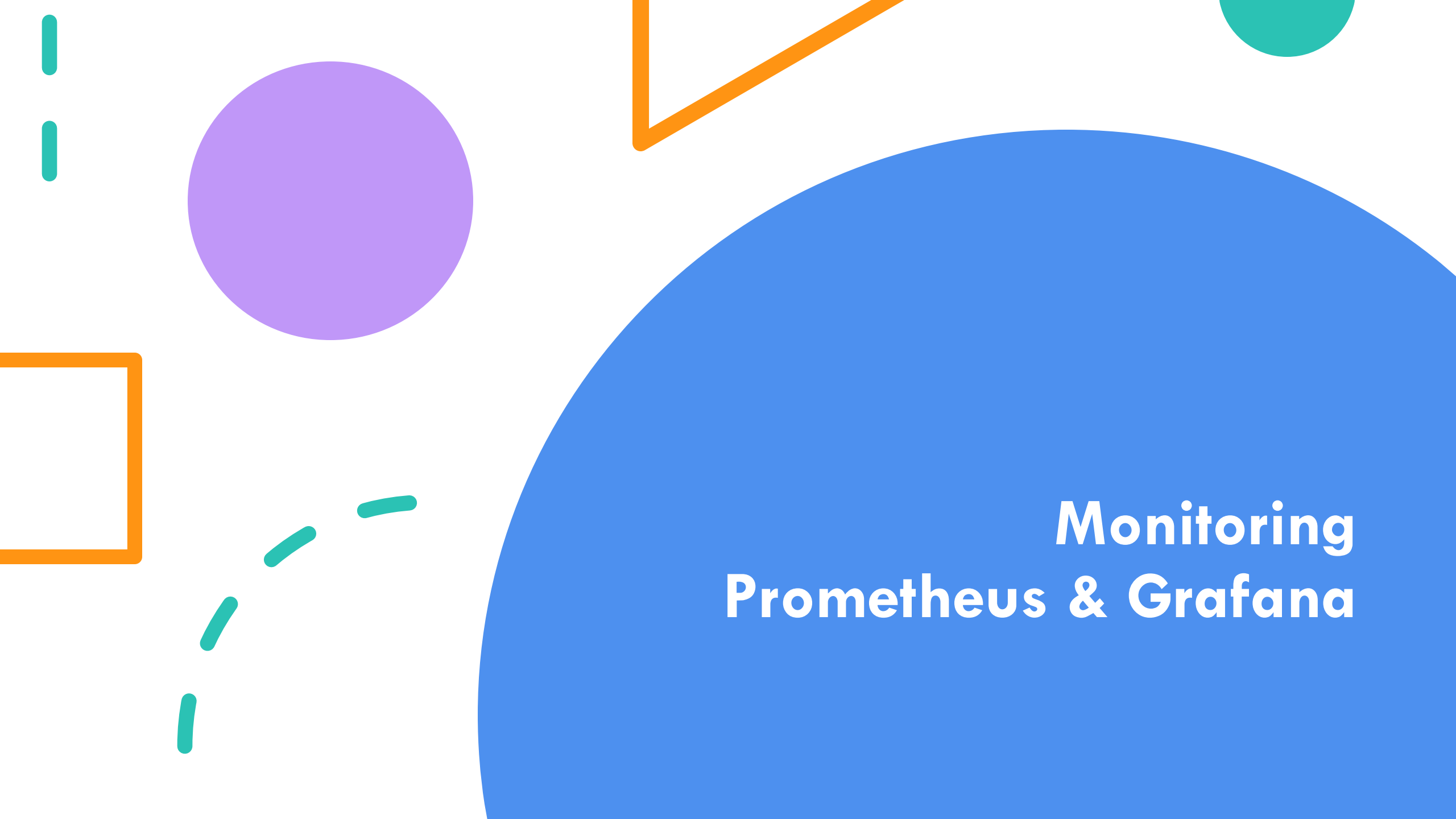


The background features a large blue circle on the right side. To its left is a purple circle. Further left are two vertical teal dashes and an orange square outline. Below the purple circle are three teal dashes. In the top right corner, there is a teal circle and an orange line forming a corner.

Monitoring Prometheus & Grafana

A large blue circle on the left side of the slide, with a smaller purple circle at its bottom-left edge.

Today's Lesson

Monitoring and observability concepts

Prometheus introduction

Grafana introduction

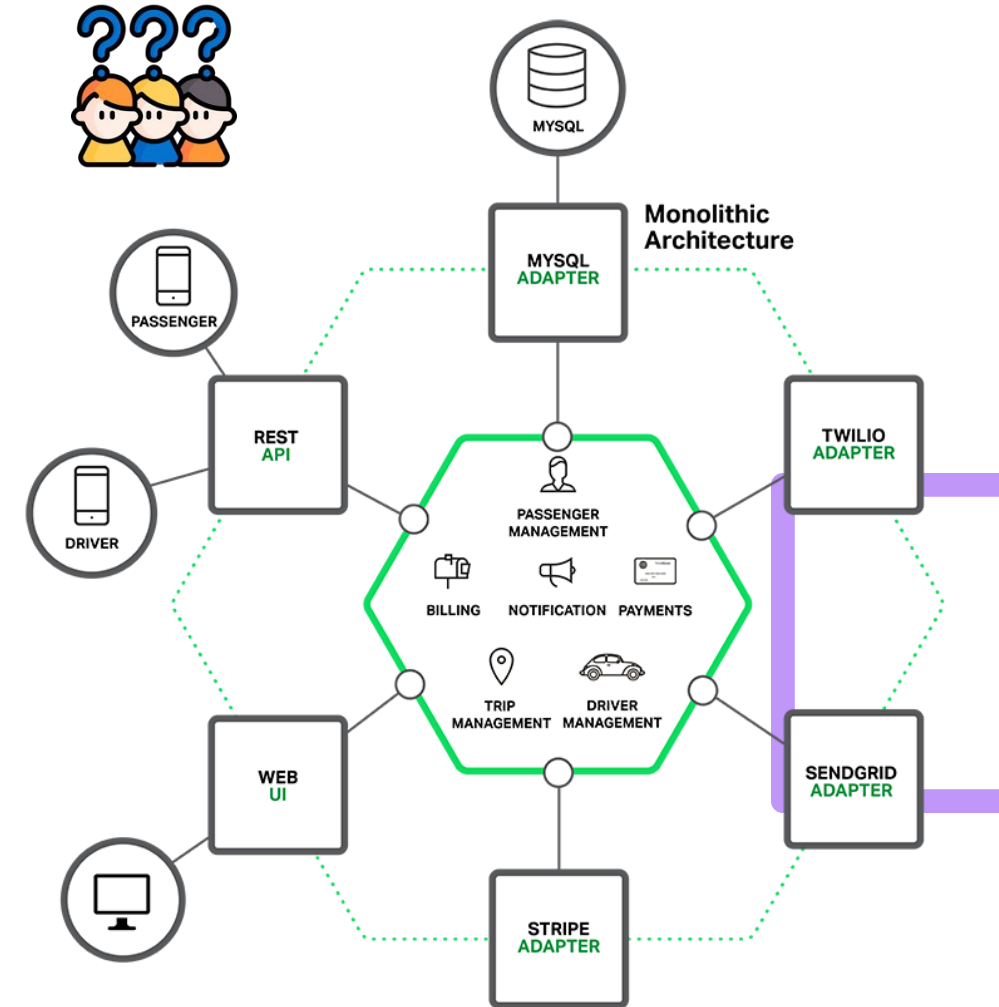


Monitoring & observability concepts

The Problem

- We have multiple software running on production. Without monitoring:

Area	Without Monitoring/Observability
Detection	You find out about problems late
Diagnosis	You can't tell why problems happened
Resolution	Fixes take longer and may be wrong
Prevention	Can't predict or prevent future issues
Business Impact	Poor uptime, user dissatisfaction, lost revenue
Culture Impact	Fear of deployments, blame culture returns



Why Monitoring & Observability Matter

- In DevOps, teams aim for continuous delivery and reliable systems in production. As systems grow more complex (microservices, cloud-native, distributed architectures), traditional monitoring isn't always enough to understand what's *actually* happening inside the system. That's where **Observability** extends those capabilities.
- Think of monitoring as "knowing when something is wrong."
- Key Concepts:
 - **Metrics:** Numeric indicators of system health (e.g., CPU usage, request latency, error rate).
 - **Logs:** Time-stamped records of discrete events in the system.
 - **Alerts:** Notifications triggered when metrics cross predefined thresholds.
 - **Dashboards:** Visual displays of key metrics for real-time system view.

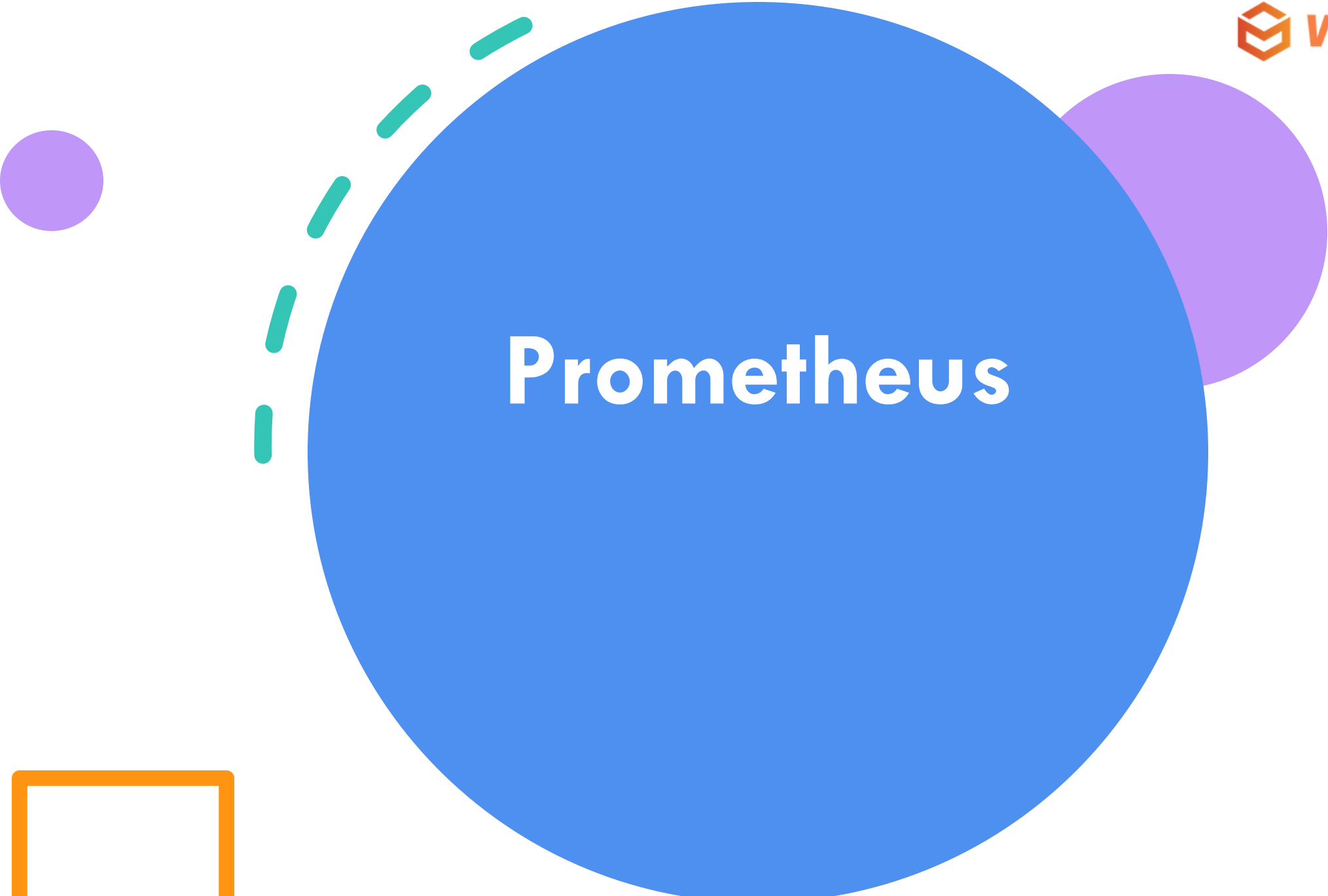
Monitoring & Observability implementation

- High-Level Implementation Steps:
 - Define goals & identify critical metrics/SLOs.
 - Instrument systems for metrics, logs, and traces.
 - Deploy data collection & visualization tools.
 - Configure actionable alerts and dashboards.
 - Create feedback loops for continuous improvement.
 - Foster a DevOps culture where observability is everyone's responsibility.

Monitoring & Observability implementation

- **In-short:** As a DevOps engineer, I want to implement a centralized monitoring system that provides unified visibility into all services and infrastructure, so the team can easily understand system health and quickly respond to incidents.

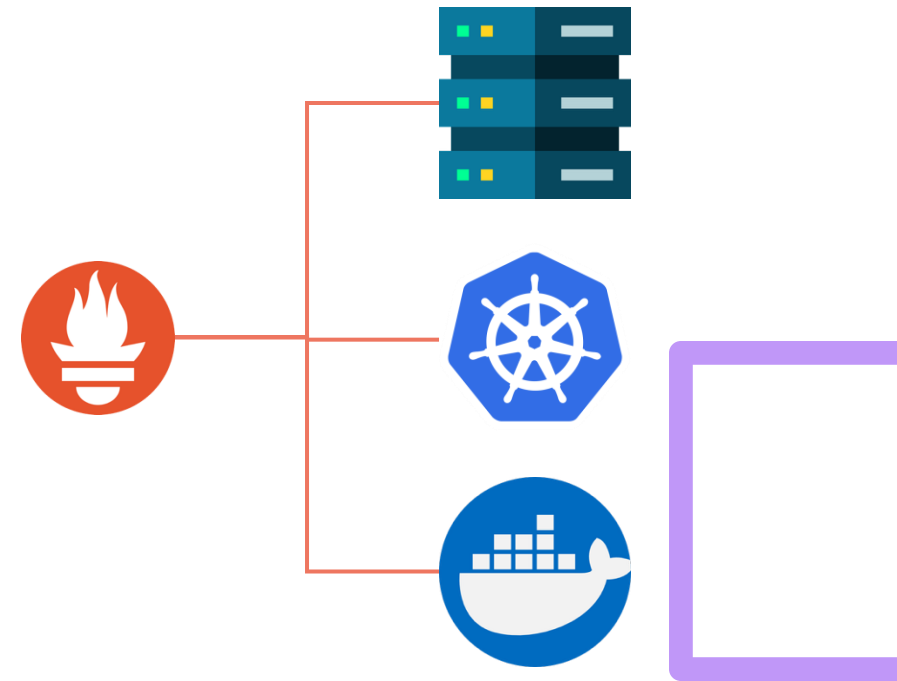




Prometheus

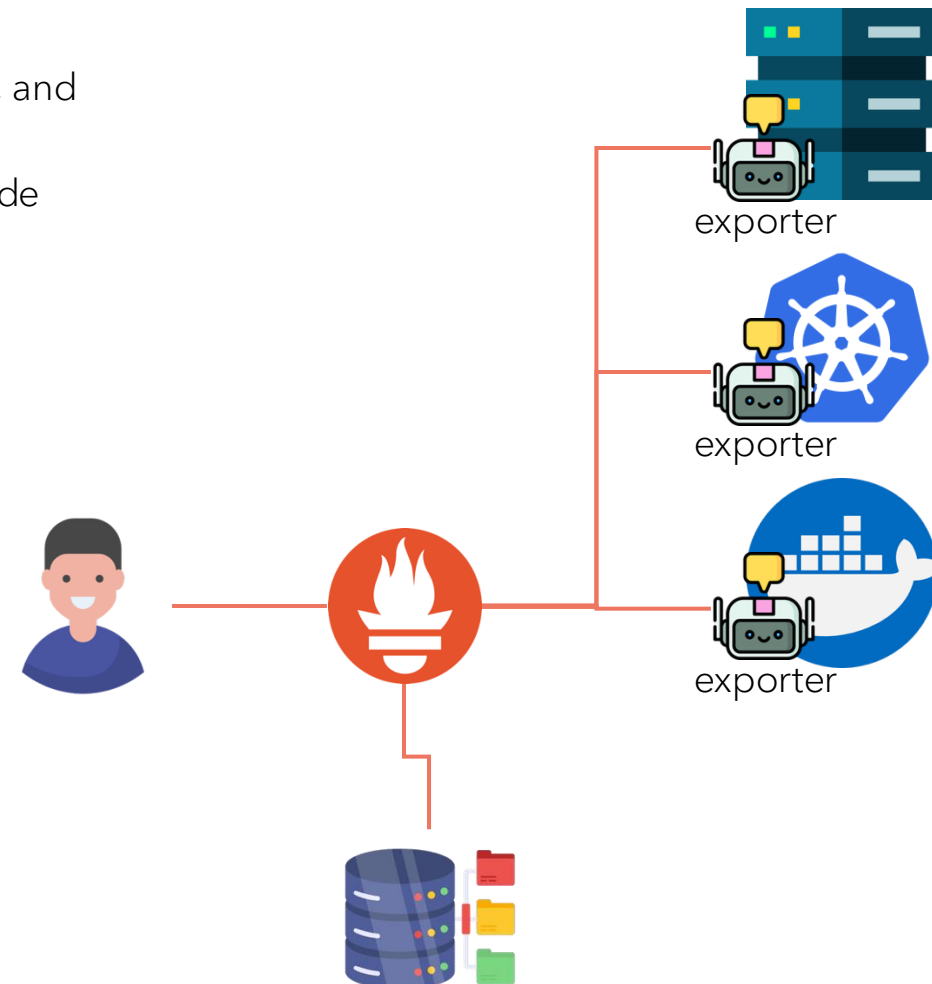
Prometheus

- Prometheus is an **open-source monitoring and alerting toolkit** designed for **recording time-series data** (metrics over time).
- Prometheus can collect metric from multiple platform like: Server, VM, Docker, Kubernetes



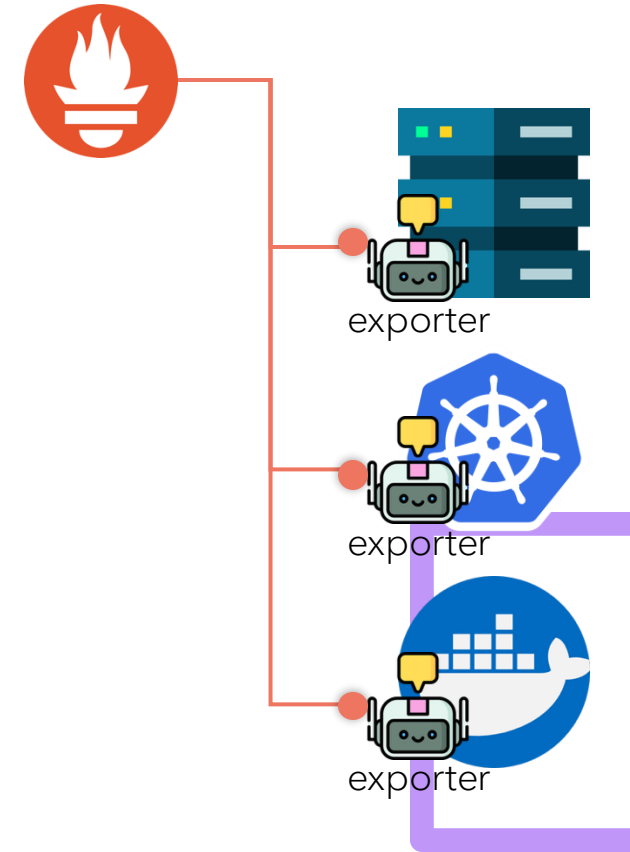
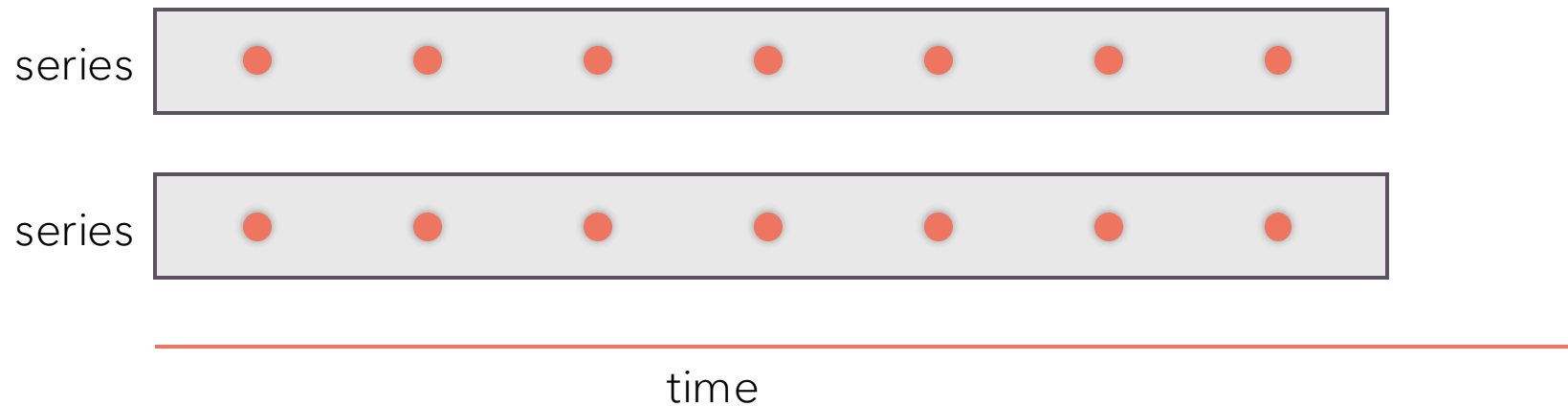
Prometheus Architecture

- Core Components:
 - **Prometheus Server** – Scrapes metrics, stores time-series data, and processes queries.
 - **Exporters** – Expose metrics from systems or applications (Node Exporter, MySQL Exporter, etc.).
- **Architecture Flow:** Metrics → Scraped by Server → Stored in TSDB → Queried via PromQL

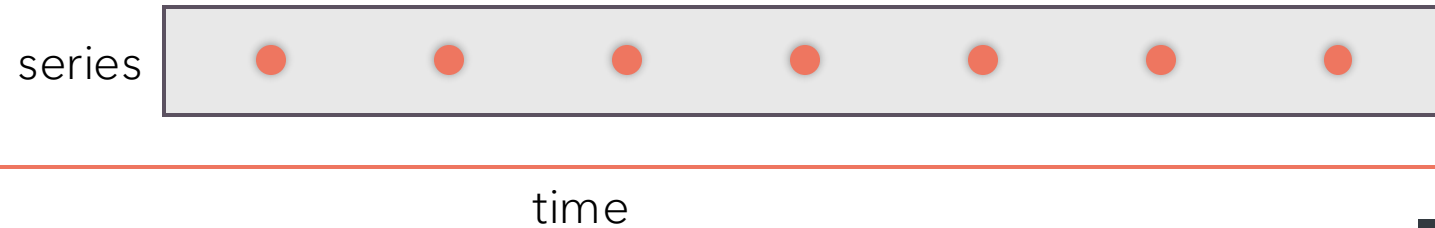


Prometheus Architecture

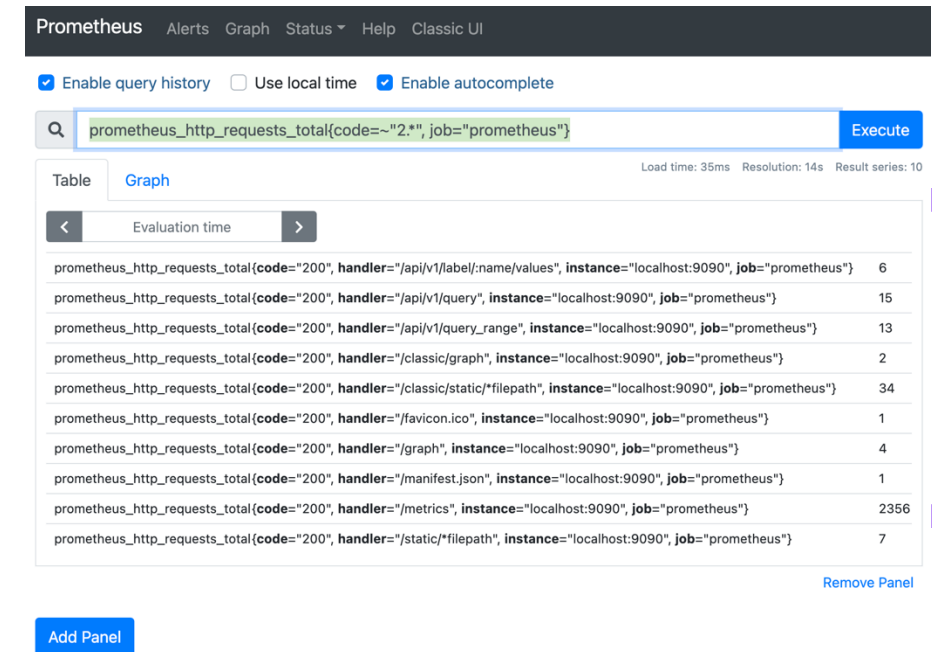
- Prometheus will pull metric from **target** by interval (default every 30 second)



PromQL (Prometheus Query Language)



- Metrics are stored as a **series** of timestamped values with labels attached.
- `http_requests_total{method="GET", status="200"} 152`
 - **Metric name:** `http_requests_total`
 - **Labels:** key-value pairs (e.g., `method`, `status`)
 - **Value:** numerical measurement
 - **Timestamp:** exact time metric recorded



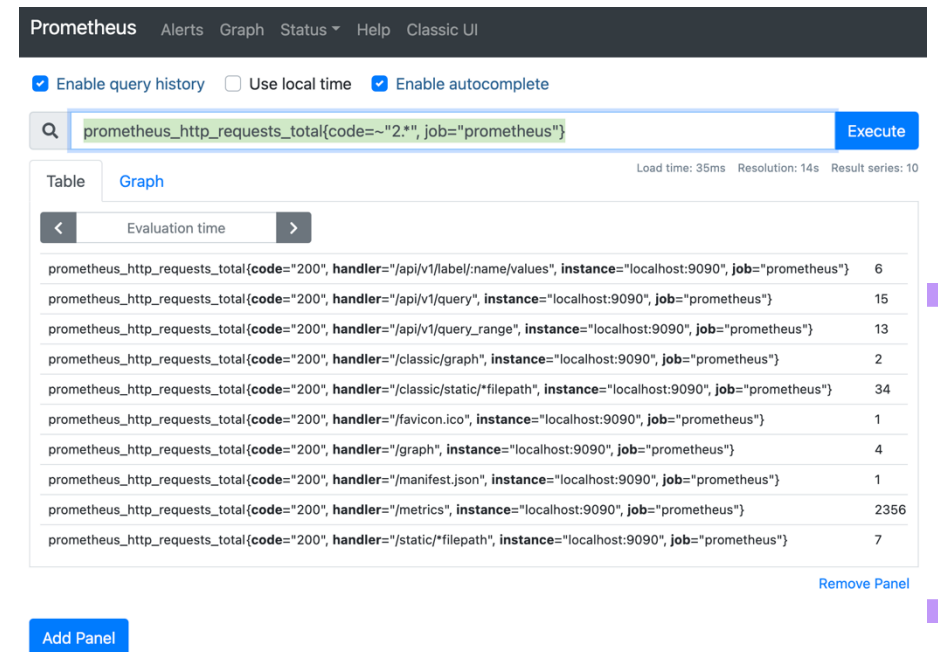
The screenshot shows the Prometheus web interface. At the top, there's a navigation bar with "Prometheus", "Alerts", "Graph", "Status", "Help", and "Classic UI". Below this, there are checkboxes for "Enable query history", "Use local time", and "Enable autocomplete". A search bar contains the query `prometheus_http_requests_total{code=~"2.*", job="prometheus"}` and an "Execute" button. Below the search bar, there's a "Table" tab and a "Graph" tab. The "Table" tab is active, showing a table of results. The table has columns for the metric name, labels, and value. The results are as follows:

Metric Name	Labels	Value
<code>prometheus_http_requests_total</code>	<code>{code="200", handler="/api/v1/label/name/values", instance="localhost:9090", job="prometheus"}</code>	6
<code>prometheus_http_requests_total</code>	<code>{code="200", handler="/api/v1/query", instance="localhost:9090", job="prometheus"}</code>	15
<code>prometheus_http_requests_total</code>	<code>{code="200", handler="/api/v1/query_range", instance="localhost:9090", job="prometheus"}</code>	13
<code>prometheus_http_requests_total</code>	<code>{code="200", handler="/classic/graph", instance="localhost:9090", job="prometheus"}</code>	2
<code>prometheus_http_requests_total</code>	<code>{code="200", handler="/classic/static/filepath", instance="localhost:9090", job="prometheus"}</code>	34
<code>prometheus_http_requests_total</code>	<code>{code="200", handler="/favicon.ico", instance="localhost:9090", job="prometheus"}</code>	1
<code>prometheus_http_requests_total</code>	<code>{code="200", handler="/graph", instance="localhost:9090", job="prometheus"}</code>	4
<code>prometheus_http_requests_total</code>	<code>{code="200", handler="/manifest.json", instance="localhost:9090", job="prometheus"}</code>	1
<code>prometheus_http_requests_total</code>	<code>{code="200", handler="/metrics", instance="localhost:9090", job="prometheus"}</code>	2356
<code>prometheus_http_requests_total</code>	<code>{code="200", handler="/static/filepath", instance="localhost:9090", job="prometheus"}</code>	7

At the bottom of the interface, there's an "Add Panel" button and a "Remove Panel" link.

PromQL (Prometheus Query Language)

- Prometheus provides a functional query language called PromQL (Prometheus Query Language) that lets the user select and aggregate time series data in real time.
- PromQL data type
 - **Instant vector** - a set of time series containing a single sample for each time series, all sharing the same timestamp
 - **Range vector** - a set of time series containing a range of data points over time for each time series
 - **Scalar** - a simple numeric floating point value
 - **String** - a simple string value; currently unused



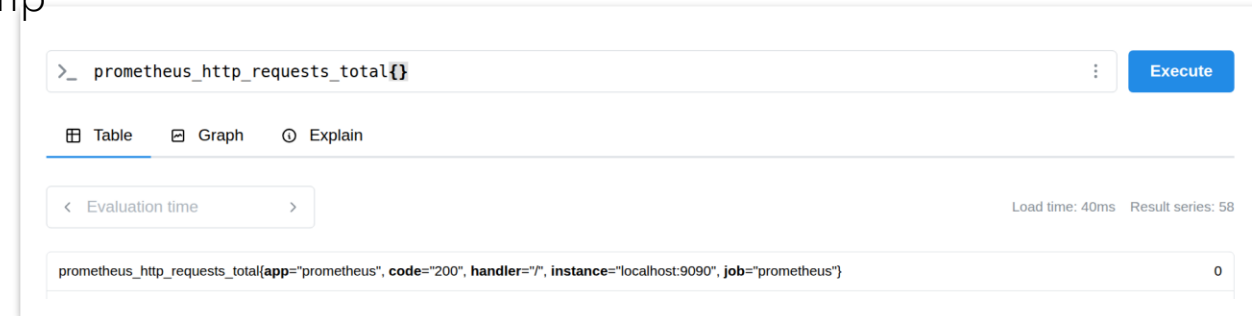
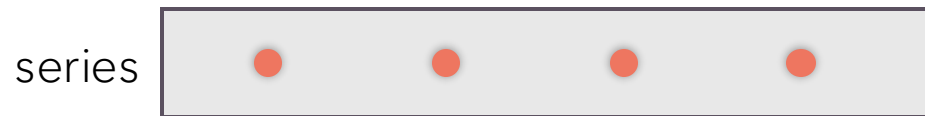
The screenshot shows the Prometheus web interface. At the top, there's a navigation bar with 'Prometheus', 'Alerts', 'Graph', 'Status', 'Help', and 'Classic UI'. Below this, there are checkboxes for 'Enable query history', 'Use local time', and 'Enable autocomplete'. A search bar contains the query 'prometheus_http_requests_total{code=~"2.*", job="prometheus"}' and an 'Execute' button. Below the search bar, there are tabs for 'Table' and 'Graph'. The 'Table' tab is selected, showing a table of results. The table has columns for the metric name, labels, and a value. The results are for the 'prometheus_http_requests_total' metric, filtered by 'code' and 'job'.

metric	value
prometheus_http_requests_total{code="200", handler="/api/v1/label/name/values", instance="localhost:9090", job="prometheus"}	6
prometheus_http_requests_total{code="200", handler="/api/v1/query", instance="localhost:9090", job="prometheus"}	15
prometheus_http_requests_total{code="200", handler="/api/v1/query_range", instance="localhost:9090", job="prometheus"}	13
prometheus_http_requests_total{code="200", handler="/classic/graph", instance="localhost:9090", job="prometheus"}	2
prometheus_http_requests_total{code="200", handler="/classic/static/*filepath", instance="localhost:9090", job="prometheus"}	34
prometheus_http_requests_total{code="200", handler="/favicon.ico", instance="localhost:9090", job="prometheus"}	1
prometheus_http_requests_total{code="200", handler="/graph", instance="localhost:9090", job="prometheus"}	4
prometheus_http_requests_total{code="200", handler="/manifest.json", instance="localhost:9090", job="prometheus"}	1
prometheus_http_requests_total{code="200", handler="/metrics", instance="localhost:9090", job="prometheus"}	2356
prometheus_http_requests_total{code="200", handler="/static/*filepath", instance="localhost:9090", job="prometheus"}	7

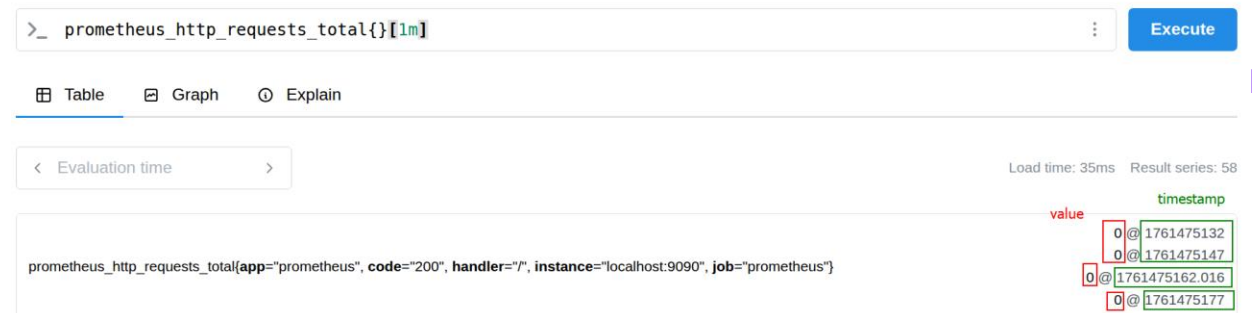
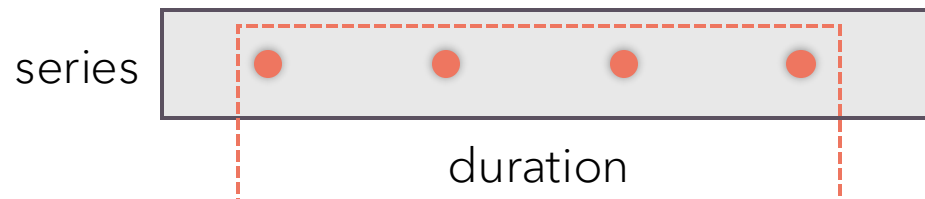
At the bottom of the interface, there are buttons for 'Add Panel' and 'Remove Panel'.

PromQL (Prometheus Query Language)

- **Instant vector** - a set of time series containing a single sample for each time series, all sharing the same timestamp



- **Range vector** - a set of time series containing a range of data points over time for each time series



PromQL Label selector

This example selects only those time series with the `prometheus_http_requests_total` metric name that also have the job label set to `prometheus` and their group label set to `canary`:

- `prometheus_http_requests_total{app="prometheus", code=~"2.*"}`

It is also possible to negatively match a label value, or to match label values against regular expressions. The following label matching operators exist:

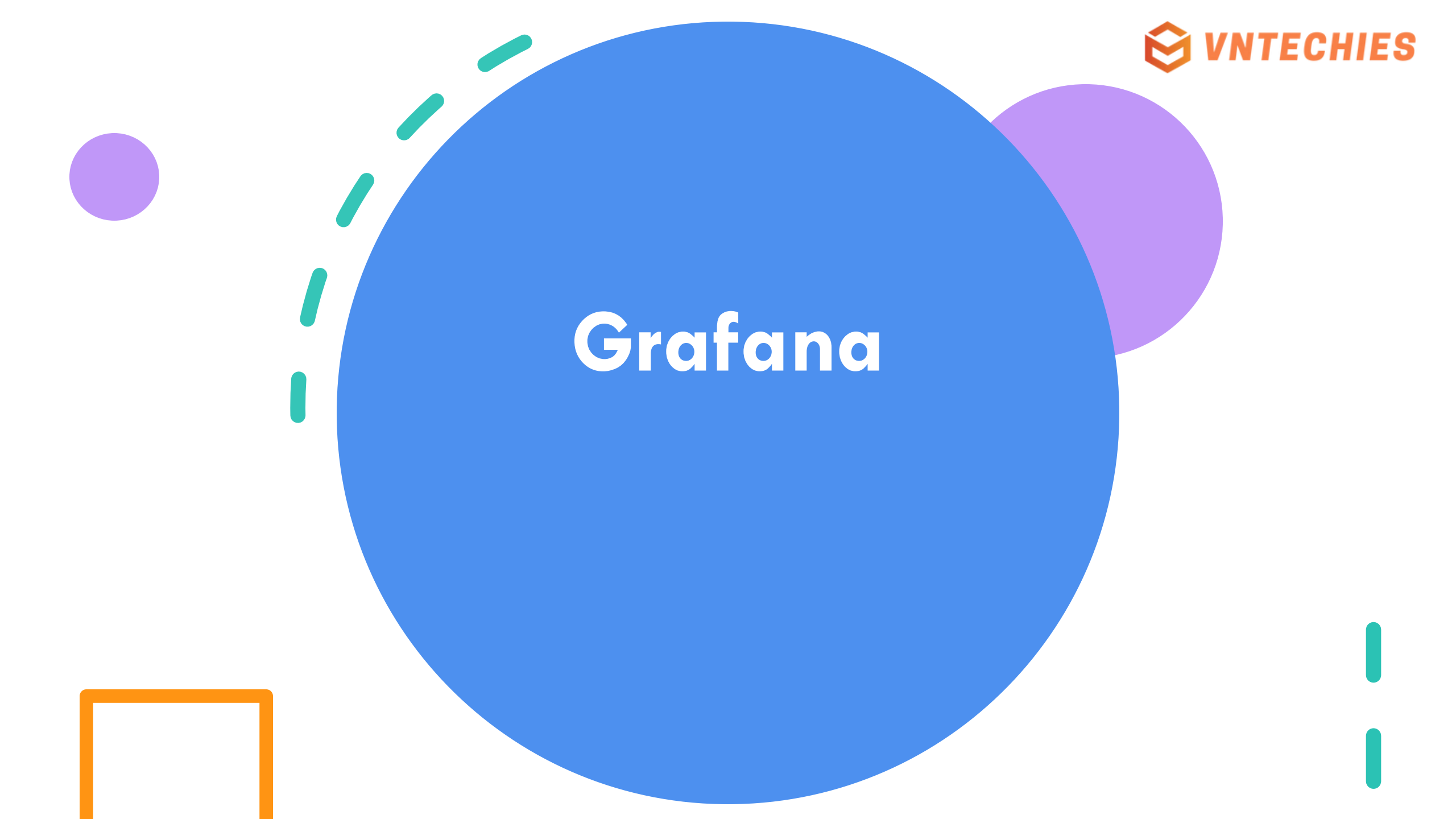
- `=`: Select labels that are exactly equal to the provided string.
- `!=`: Select labels that are not equal to the provided string.
- `=~`: Select labels that regex-match the provided string.
- `!~`: Select labels that do not regex-match the provided string.

Regex matches are fully anchored. A match of `env=~"foo"` is treated as `env=~"^foo$"`.

PromQL Function

Prometheus provided multiple function to extend feature of time series metric

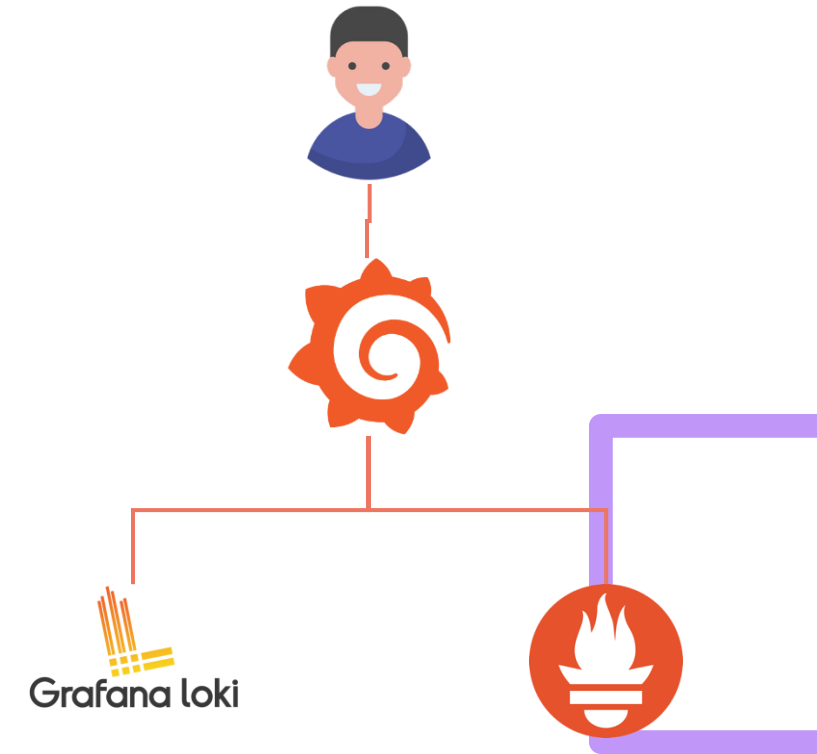
Function	Purpose	Example
<code>rate()</code>	Rate of change for counters over time.	<code>rate(http_requests_total[5m])</code>
<code>increase()</code>	Total increase over a time period.	<code>increase(errors_total[1h])</code>
<code>sum()</code>	Aggregate multiple series into one.	<code>sum(cpu_usage)</code>
<code>avg()</code>	Calculate average value.	<code>avg(node_memory_usage_bytes)</code>
<code>max()</code> , <code>min()</code>	Find maximum or minimum.	<code>max(cpu_usage)</code> , <code>min(up)</code>
<code>histogram_quantile()</code>	Calculate quantile (e.g., p90 latency).	<code>histogram_quantile(0.9, rate(request_duration_seconds_bucket[5m]))</code>



Grafana

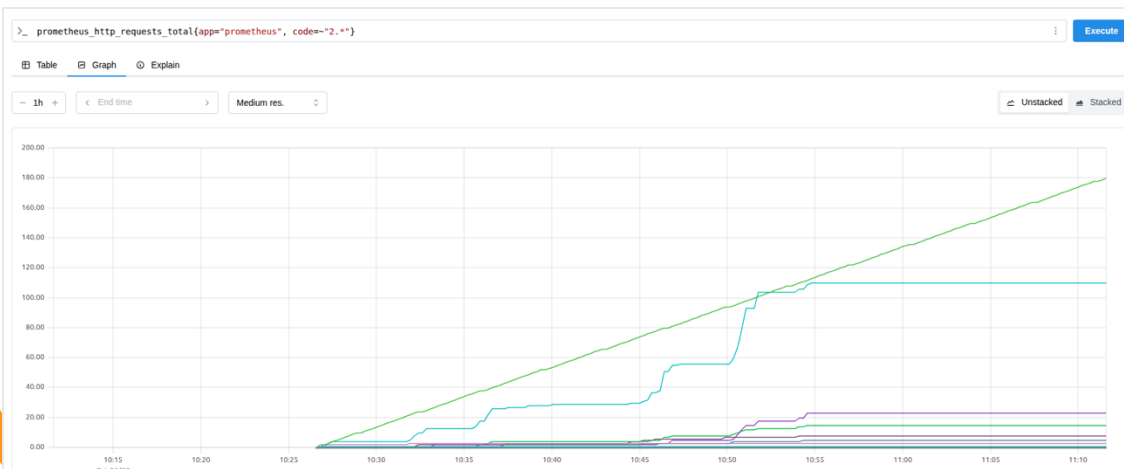
What is Grafana

- Grafana = Visualization platform that connects to Prometheus and other data sources.
- Key Features:
 - Real-time interactive dashboards.
 - Query editor for PromQL.
 - Supports team sharing, permissions, and alerting.



Grafana

- Prometheus + Grafana Integration
 - Prometheus = data collection and storage.
 - Grafana = visualization and dashboarding.
- Workflow:
 - Prometheus scrapes metrics.
 - Grafana queries Prometheus data.
 - Dashboards display live system health.
 - Optional alerts sent through Grafana or Alertmanager.



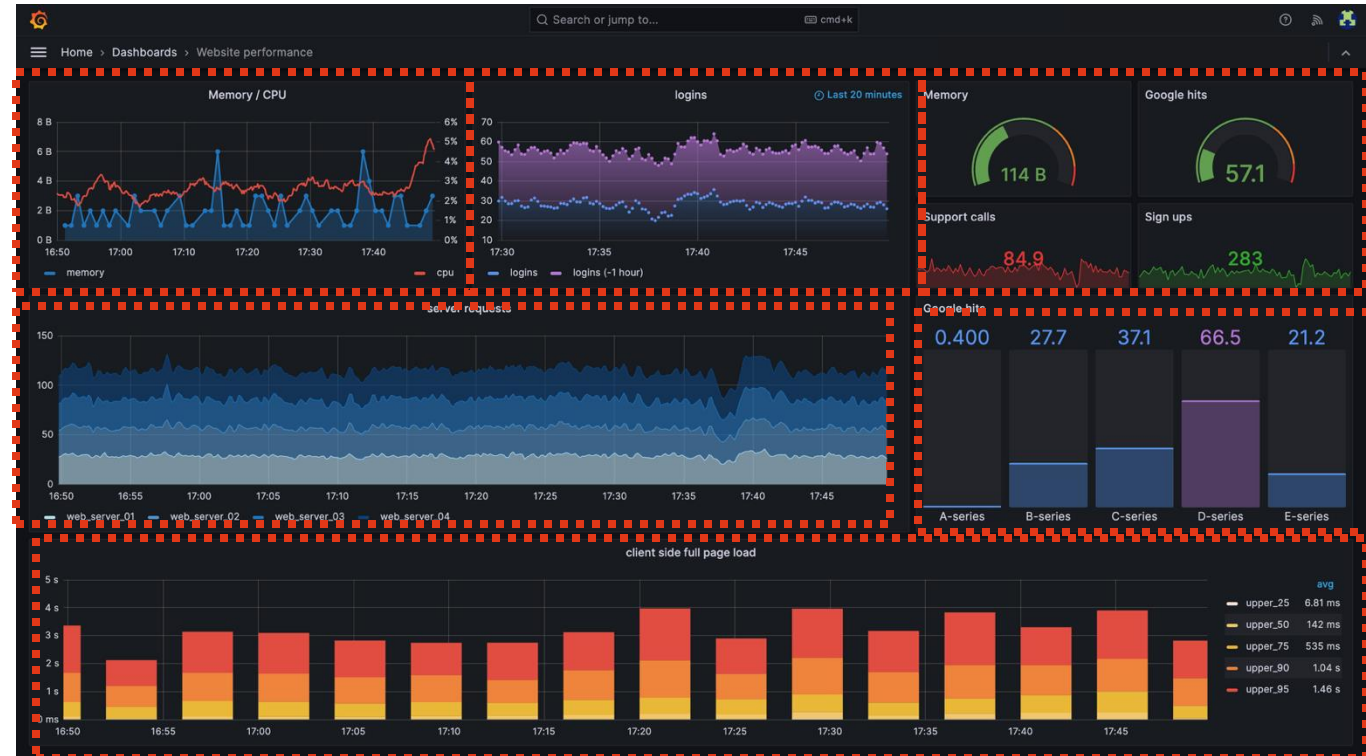
Grafana Dashboard

- A **dashboard** is a collection of **panels** that display metrics from one or more data sources (like Prometheus).
- Used to monitor **system performance**, **application metrics**, or **business KPIs** in real time.
- Dashboards can be shared, customized, and templated.
- Each dashboard can show multiple aspects of your system – CPU, memory, network, errors, latency, etc.



Grafana Panel

- A **panel** is an individual visualization within a dashboard.
- Each panel runs a query (e.g., a PromQL expression) and displays the result as a graph, table, gauge, heatmap, etc.





Practice Lab